PROGRAMACIÓN III

AVANCE DEL PROYECTO · PROGRESO 1

SIGREL

Sistema Inteligente de Gestión de Reparaciones de Laptops

Asignatura: Programación III · Estructuras de Datos y Algoritmos en Java

Docente: Ing. Paulo Guerra T.

Estudiante: Andrés Valarezo · Proyecto individual

Fecha: Quito, 18 de octubre de 2026

Estado: PROGRESO 1 · Análisis, diseño y selección de la solución

Resumen de la propuesta

SIGREL es un diseño preliminar de software orientado a objetos en Java para gestionar órdenes de servicio de laptops. Su contribución algorítmica es organizar la cola de trabajos usando urgencia y tiempo de espera, recuperar órdenes rápidamente por ID y apoyar asignaciones según especialidad y capacidad del técnico. Se comparan un enfoque de listas con otro híbrido basado en HashMap, PriorityQueue y ArrayList; se selecciona este último bajo supuestos explícitos. El documento presenta cinco módulos funcionales, modelo UML editable, un plan de trabajo y criterios de verificación. No se afirma que exista una implementación funcional ni resultados experimentales en esta etapa.

1. Formulación del problema

En los talleres que reciben laptops para diagnóstico y reparación, una misma jornada puede incluir fallas muy distintas: equipos que no encienden, pantallas rotas, baterías degradadas y problemas de carga. El problema de gestión aparece cuando cada ingreso debe identificarse, evaluarse, ordenarse y seguirse hasta la devolución al cliente. Para este proyecto se considera un taller pequeño hipotético en Quito. Es un escenario de diseño, no el resultado de una encuesta ni un diagnóstico verificado de un negocio concreto. Se supone que el taller atiende varias órdenes simultáneas y que dispone de un número limitado de técnicos con distintas especialidades.

Cuando los datos de clientes, equipos, fallas y estados se anotan en medios dispersos, surgen riesgos de duplicación, búsquedas lentas y pérdida de trazabilidad. El personal de recepción necesita localizar una orden mediante su identificador o el número de serie de la laptop; el técnico requiere conocer el diagnóstico y las tareas pendientes; el responsable del taller necesita decidir qué trabajo asignar a quién. Si la prioridad se determina únicamente por la llegada, un equipo con una falla urgente podría esperar detrás de trabajos menos críticos. Por el contrario, privilegiar siempre a las órdenes nuevas y urgentes puede postergar indefinidamente aquellas que llevan más tiempo en cola.

La literatura demuestra que la secuenciación es importante en servicios de reparación. Srivathsan y Viswanathan (2017) estudian conjuntamente la capacidad de reparación y la disponibilidad de componentes, mostrando que ambas decisiones influyen en el funcionamiento de un centro de servicio. Más recientemente, Wulanda et al. (2026) presentan un sistema para gestionar reparaciones informáticas basado en Shortest Job First y un mecanismo de envejecimiento, con el propósito de reducir demoras y evitar la postergación de ciertos trabajos. Estos estudios fundamentan la pertinencia de analizar reglas de espera y asignación, pero no significan que SIGREL haya demostrado todavía una reducción real de tiempos.

Los actores del problema son: clientes que entregan y retiran sus equipos; personal de recepción que registra datos y consulta avances; técnicos que inspeccionan y reparan; y un administrador que revisa carga de trabajo y asignaciones. La información principal comprende identificadores de clientes y equipos, números de serie, descripciones de fallas, fechas de ingreso, categorías de urgencia, estados, diagnósticos, especialidades de técnicos, capacidad operativa e historial cronológico de eventos. Cada transición de estado modifica lo que puede hacerse con una orden; por ejemplo, una orden entregada ya no debe reaparecer como pendiente.

Entre las causas que el proyecto busca abordar están la ausencia de reglas uniformes para clasificar incidencias, la dificultad para recuperar información con rapidez y la asignación de reparaciones sin comprobar especialidad ni disponibilidad. Sus consecuencias potenciales incluyen demoras de atención, cargas desequilibradas, comunicaciones inconsistentes y falta de evidencia acerca de las

decisiones realizadas. El sistema también enfrenta restricciones: debe construirse en Java; empleará estructuras de datos y algoritmos visibles; tendrá un alcance realizable en un curso; y no asumirá integración automática con proveedores, sistemas de facturación ni herramientas de diagnóstico físico.

Los criterios del sistema deberán ser explícitos y comprobables. Se propone clasificar la urgencia en baja, media o alta; registrar la fecha de ingreso; aumentar gradualmente el puntaje por días transcurridos; y utilizar reglas de desempate estables. Una orden solamente podrá asignarse a un técnico si su especialidad es compatible con el equipo y tiene cupo disponible. El seguimiento deberá preservar eventos sucesivos de manera que sea posible reconstruir qué ocurrió, cuándo y en qué estado quedó el trabajo. Se usarán datos de prueba ficticios para no exponer información personal de clientes reales.

El contexto tiene además una dimensión económica y ambiental justificable. El informe Global E-waste Monitor 2024 indica que en 2022 se generaron 62 millones de toneladas de residuos electrónicos a escala mundial y solo el 22,3 % fue documentado como recogido y reciclado formalmente (ITU y UNITAR, 2024). Este dato no corresponde específicamente a laptops o talleres de Quito, pero aporta contexto a la importancia de prolongar la vida útil de dispositivos electrónicos. En una línea similar, la Comisión Europea (2024) adoptó normas orientadas a favorecer la reparación y el consumo sostenible en su jurisdicción; estas normas se citan como referencia internacional, no como legislación ecuatoriana aplicable al taller propuesto.

El problema computacional queda planteado así: ¿cómo organizar y recuperar eficientemente la información de las órdenes de reparación, priorizar los trabajos con reglas transparentes y asignarlos a técnicos disponibles sin perder la trazabilidad de las decisiones? La solución deberá contrastar al menos dos formas de almacenamiento y ordenamiento, evaluar su costo temporal y espacial, y seleccionar una alternativa que equilibre rapidez, memoria y facilidad de desarrollo. En esta primera fase se formula y diseña el sistema; las implementaciones funcionales, mediciones y pruebas quedarán para los avances posteriores.

2. Alcance y módulos funcionales

2.1 Objetivo y límites

Objetivo general: diseñar un sistema de gestión de reparaciones que facilite la búsqueda, clasificación, priorización, asignación y trazabilidad de órdenes mediante estructuras de datos y algoritmos en Java.

Alcance del prototipo futuro: aplicación local en Java con datos de prueba ficticios; no incluye autenticación productiva, pasarelas de pago, facturación, notificaciones automáticas, diagnóstico de hardware ni conexión en tiempo real con proveedores. La persistencia es una mejora posible posterior, no requisito asumido del Progreso 1.

M1. Clientes y equipos

Propósito: mantener la relación entre clientes y laptops. Actor principal: Recepcionista.

Datos: Cliente, contacto, Equipo, número de serie, marca, modelo.

Operaciones (mínimo cuatro): Registrar clientes con datos mínimos; asociar laptops a un cliente; buscar equipos por serie; filtrar equipos por marca o cliente.

Reglas: Una serie identifica a un equipo dentro del sistema; un cliente puede asociar múltiples equipos; la búsqueda no debe devolver información de terceros sin autorización.

Relación con otros módulos: Entrega los equipos a M2; consulta órdenes desde M5.

Responsable: Andrés Valarezo (desarrollo individual).

M2. Órdenes y clasificación

Propósito: controlar el ingreso y clasificación de trabajos. Actor principal: Recepcionista.

Datos: OrdenReparacion, fechas, falla reportada, urgencia, estado inicial.

Operaciones (mínimo cuatro): Abrir una orden; validar que no exista otra orden activa para el mismo equipo; clasificar la urgencia; listar órdenes por estado y fecha.

Reglas: Las órdenes poseen ID único; reciben estado RECIBIDA; la urgencia es 1 (baja), 2 (media) o 3 (alta); no se abre otra orden activa del mismo equipo.

Relación con otros módulos: Recibe Equipo de M1; alimenta la cola M3 y el historial M5.

Responsable: Andrés Valarezo (desarrollo individual).

M3. Priorización de reparaciones

Propósito: organizar qué orden debe revisarse primero. Actor principal: Administrador.

Datos: Órdenes pendientes, nivel de urgencia, días de espera, puntaje.

Operaciones (mínimo cuatro): Calcular puntaje; encolar órdenes elegibles; consultar la siguiente prioridad; recalcular y reconstruir la cola cuando cambia el día o la urgencia.

Reglas: Puntaje = 10 × urgencia + 2 × días de espera; mayor puntaje primero; empate por ingreso más antiguo y luego ID; solo órdenes pendientes.

Relación con otros módulos: Recibe órdenes M2; suministra candidatos de trabajo a M4; consulta estados M5.

Responsable: Andrés Valarezo (desarrollo individual).

M4. Técnicos y asignaciones

Propósito: distribuir trabajo entre técnicos elegibles. Actor principal: Administrador.

Datos: Tecnico, especialidades, capacidad, Asignacion y carga activa.

Operaciones (mínimo cuatro): Identificar técnicos compatibles; filtrar técnicos con cupo; elegir al de menor carga; asignar o reasignar una orden.

Reglas: Una orden tiene como máximo una asignación activa; no superar capacidadMax; las reasignaciones conservan el historial de asignaciones.

Relación con otros módulos: Consume pendientes M3; produce eventos de asignación hacia M5.

Responsable: Andrés Valarezo (desarrollo individual).

M5. Diagnóstico y seguimiento

Propósito: registrar diagnósticos, estados y evidencia de seguimiento. Actor principal: Técnico.

Datos: Diagnostico, costo estimado, estados, EventoSeguimiento, observaciones.

Operaciones (mínimo cuatro): Registrar diagnóstico; cambiar estado de una orden; mostrar historial cronológico; calcular días de espera y órdenes por estado.

Reglas: Transiciones válidas: RECIBIDA→EN_DIAGNOSTICO→EN_REPARACION→LISTA→ENTREGADA; CANCELADA como cierre permitido antes de entrega; toda transición queda registrada.

Relación con otros módulos: Actualiza órdenes M2; excluye las cerradas de M3; informa disponibilidad a M4.

Responsable: Andrés Valarezo (desarrollo individual).

3. Modelo orientado a objetos y UML preliminar

Se proponen siete clases de dominio: Cliente, Equipo, OrdenReparacion, Diagnostico, Tecnico, Asignacion y EventoSeguimiento, además de la enumeración EstadoOrden. El módulo de priorización se implementaría posteriormente como servicio sobre las entidades, sin convertirse artificialmente en una clase de dominio.

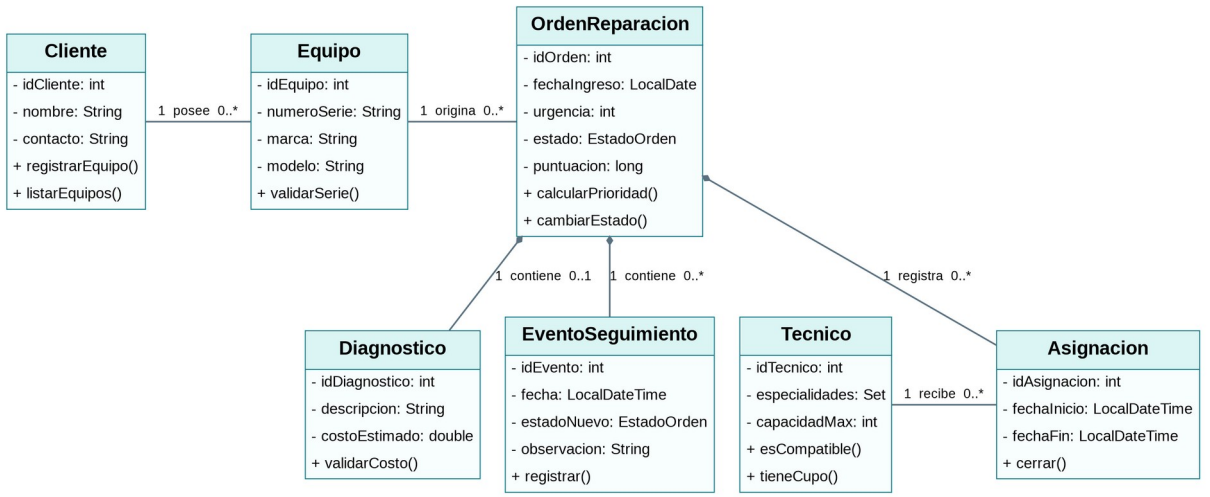


Figura 1. Diagrama UML preliminar de SIGREL. Se adjuntan los archivos editables docs/uml/modelo_sigrel.puml y docs/uml/modelo_sigrel.dot para su inspección, edición y defensa técnica.

3.1 Relaciones y multiplicidades

- Cliente (1) – Equipo (0..*): un cliente puede poseer varios equipos; cada equipo pertenece a un cliente.
- Equipo (1) – OrdenReparacion (0..*): una laptop puede generar diferentes servicios a lo largo del tiempo.
- OrdenReparacion (1) – Diagnostico (0..1): una orden puede contener un diagnóstico, pendiente hasta la revisión.
- OrdenReparacion (1) – EventoSeguimiento (0..*): composición porque cada evento pertenece a una orden.
- OrdenReparacion (1) – Asignacion (0..*): se conserva el historial de asignaciones; solo una activa.
- Tecnico (1) – Asignacion (0..*): un técnico puede tener asignaciones, con límite de capacidad.

3.2 Consideraciones de consistencia

OrdenReparacion controla las transiciones de estado; el gestor de colas solo conserva órdenes elegibles. Las fechas de los eventos no se modifican después de registrarse. Para evitar duplicados, se consultan las órdenes activas antes de registrar un nuevo ingreso. En futuras iteraciones convendría validar las clases y multiplicidades con al menos un caso de uso por módulo.

4. Alternativas de estructuras de datos y algoritmos

Variables: n = número de órdenes almacenadas, p = número de órdenes pendientes, t = número de técnicos, e = número de eventos por orden. Se comparan los costos asintóticos, no tiempos medidos.

Opción	Estructuras	Algoritmos / operaciones	Complejidad temporal	Espacio / límites
A. Listas y ordenamiento	ArrayList de órdenes, técnicos y eventos	Búsqueda secuencial por ID; ordenar pendientes al consultar siguiente; recorrer técnicos compatibles.	Insertar O(1) amort.; buscar O(n); siguiente ordenado O(p log p); técnicos O(t).	O(n+t+e). Implementación muy simple; repetición de búsquedas y ordenamientos.
B. Índice + heap (elegida)	HashMap<ID,Orden>; PriorityQueue<Orden> (heap); ArrayList<Evento>	Búsqueda indexada; offer/poll por prioridad; reconstrucción diaria por envejecimiento; filtrado de técnicos.	Buscar O(1) promedio; offer/poll O(log p); rehacer heap O(p); técnicos O(t).	O(n+p+t+e). Más memoria y sincronización índice/cola; mejor consulta reiterada.

4.1 Regla de prioridad propuesta

Puntaje = 10 × urgencia + 2 × días completos desde el ingreso, donde urgencia toma valores 1, 2 y 3. Se atiende primero la puntuación mayor; en empate, la orden con fecha de ingreso más antigua, luego el menor ID. Este envejecimiento incrementa la prioridad de una orden conforme espera y busca reducir su postergación. La regla es propuesta de diseño, no una fórmula empíricamente validada para talleres de Quito.

Ejemplo (misma fecha de evaluación): orden A, urgencia 3 y 0 días → 30; orden B, urgencia 2 y 3 días → 26; orden C, urgencia 1 y 12 días → 34; orden D, urgencia 3 y 2 días → 34. Orden resultante: C, D, A, B, al desempatar 34 por antigüedad.

Advertencia técnica: PriorityQueue no se reordena por cambiar campos que utiliza el comparador. Por ello, el puntaje debe congelarse al encolar y la cola reconstruirse cuando cambien el día o la urgencia; no se utilizará un comparador que dependa dinámicamente del reloj. Oracle documenta O(log p) para inserción/extracción y O(p) para búsquedas o eliminaciones por objeto (Oracle, s. f.).

5. Selección y fundamentación de la mejor alternativa

Se selecciona B porque la consulta de órdenes por identificador y la extracción repetida del siguiente trabajo son operaciones centrales del sistema. En B, HashMap permite acceso esperado O(1) con dispersión adecuada; PriorityQueue realiza inserción y extracción en O(log p), mientras que en A volver a ordenar toda la lista exige O(p log p) cada vez que se requiere una cola ordenada. ArrayList continúa siendo útil para historiales ordenados. Las complejidades se apoyan en las especificaciones de Oracle de HashMap, PriorityQueue y ArrayList.

La elección no significa que B sea superior en todas las tareas: requiere mantener coherentes el índice y el heap; una actualización de urgencia puede exigir reconstrucción O(p), y la búsqueda de

técnicos sigue siendo O(t). Para volúmenes mínimos de datos, A podría ser más sencilla y suficiente. Se propone B porque ofrece un uso auténtico de estructuras de datos del curso y mantiene una implementación factible en Java estándar.

Criterio	A. Listas	B. Índice y heap
Consulta por ID	O(n)	O(1) esperado
Nuevo pendiente	O(1) amortizado	O(log p)
Siguiente reparación	O(p log p), al reordenar	O(log p) mediante poll
Recalcular por antigüedad	O(p log p), al reordenar	O(p), reconstrucción del heap
Memoria auxiliar	Menor	Mayor: índice + heap
Mantenibilidad	Muy simple	Modular, requiere sincronización

6. Plan preliminar de implementación

6.1 Organización del trabajo individual

Rol	Componentes iniciales	Producto verificable
Etapa 1 · Responsable: estudiante	Clientes y laptops; validaciones	Modelo Cliente/Equipo; búsqueda por serie y pruebas
Etapa 2 · Responsable: estudiante	Órdenes, estados y diagnósticos	Modelo Orden/Diagnóstico; control de duplicados
Etapa 3 · Responsable: estudiante	Índice, cola y reglas de prioridad	HashMap/PriorityQueue, empates y envejecimiento
Etapa 4 · Responsable: estudiante	Técnicos y asignaciones	Disponibilidad, especialidad y pruebas
Etapa 5 · Responsable: estudiante	Seguimiento, pruebas y documentación	Historial, integración y README

Organización y responsabilidades: Andrés Valarezo es el autor y responsable de los cinco módulos de SIGREL. El plan de implementación se realizará por etapas: primero las entidades y órdenes; después las estructuras de prioridad y búsqueda; a continuación las asignaciones y el seguimiento; y finalmente las pruebas, la integración y la documentación. Esta secuencia permite controlar el alcance y avanzar de forma gradual.

6.2 Repositorio Git inicial

Carpetas propuestas: docs/uml (modelo editable y representación), docs/fuentes (bibliografía), src/main/java/ec/edu/sigrel (futura implementación) y src/test/java/ec/edu/sigrel (pruebas futuras). Archivos raíz: README.md, .gitignore y pom.xml. El archivo ZIP adjunto contiene un repositorio local con un commit inicial; todavía NO está publicado en GitHub.

6.3 Riesgos técnicos y mitigación

Riesgo	Mitigación / prueba prevista
Orden desactualizado en PriorityQueue	Recalcular puntajes y reconstruir la cola por lote; probar cambio de urgencia.
Orden duplicada en índice y cola	IDs únicos y una sola fuente de verdad para el estado de la

	orden.
Asignar técnicos sin especialidad/cupo	Validar compatibilidad y capacidad; probar casos de rechazo.
Cadenas de estados inválidas	Validación explícita de transiciones; registrar historial inmutable.
Sobrealcance del proyecto	Empezar por lógica local en Java y datos ficticios; diferir integraciones.
Carga de trabajo de cinco módulos	Dividir la implementación en etapas, definir hitos realistas y priorizar las operaciones algorítmicas esenciales.

6.4 Criterios de pruebas previstos para Progreso 2

- Registrar una laptop y recuperar su orden con ID conocido.
- Rechazar el registro de otra orden activa del mismo equipo.
- Obtener el orden C, D, A, B según el ejemplo de envejecimiento.
- Recalcular la cola tras avanzar un día sin violar el orden del heap.
- Rechazar asignar a un técnico incompatible o sin capacidad disponible.
- Conservar en secuencia todos los cambios de estado de una orden.

7. Declaración de uso de herramientas de inteligencia artificial

Herramienta	Uso realizado	Resultado incorporado	Forma de verificación
ChatGPT (OpenAI)	Apoyo en estructuración, redacción preliminar y planificación técnica	Borrador de informe, alternativas, UML y repositorio inicial	Documentación y fuentes contrastadas; revisión personal del diagrama y defensa conceptual a cargo del autor..

El uso de IA se declara de forma transparente: ChatGPT se utilizó como apoyo para organizar este borrador, proponer algoritmos, estructurar el UML y preparar documentación. El estudiante deberá verificar las referencias, editar el diagrama en una herramienta UML, comprender cada complejidad y revisar todos los materiales antes de presentarlos como propios.

8. Guion recomendado para presentación (8–10 minutos)

Minutos	Contenido	Participación
0:00–1:30	Problema, actores, variables y restricciones	Estudiante: Andrés Valarezo
1:30–3:00	Cinco módulos y alcance	Estudiante: Andrés Valarezo
3:00–4:30	Modelo UML y relaciones	Estudiante: Andrés Valarezo
4:30–6:30	Alternativas, complejidades y ejemplo	Estudiante: Andrés Valarezo
6:30–8:00	Selección de B y riesgos	Estudiante: Andrés Valarezo
8:00–9:00	Plan para Progreso 2 y próximos pasos	Estudiante: Andrés Valarezo

Referencias bibliográficas (formato APA 7)

Comisión Europea. (2024). Directive on repair of goods. https://commission.europa.eu/law/law-topic/consumer-protection-law/directive-repair-goods_en

International Telecommunication Union & United Nations Institute for Training and Research. (2024). The global e-waste monitor 2024. https://www.itu.int/pub/D-GEN-E_WASTE.01-2024

Oracle. (s. f.). Class ArrayList (Java SE 18). https://docs.oracle.com/en/java/javase/18/docs/api/java.base/java/util/ArrayList.html

Oracle. (s. f.). Class HashMap (Java SE 18). https://docs.oracle.com/en/java/javase/18/docs/api/java.base/java/util/HashMap.html

Oracle. (s. f.). Class PriorityQueue (Java SE 18). https://docs.oracle.com/en/java/javase/18/docs/api/java.base/java/util/PriorityQueue.html

Srivathsan, S., & Viswanathan, S. (2017). A queueing-based optimization model for planning inventory of repaired components in a service center. Computers & Industrial Engineering, 106, 373–385. https://doi.org/10.1016/j.cie.2017.01.020

Wulanda, N., Adrianto, S., Muhardi, M., & Fauzansyah, F. (2026). Optimizing service queue scheduling through a web-based service and repair management information system using the shortest job first algorithm with an aging mechanism. Journal of Computer-based Instructional Media, 4(1), 55–70. https://doi.org/10.58712/jcim.v4i1.162